

BUILDING AI AGENTS

A Comprehensive Technical Reference

Tools · Structured Outputs · State Management · Memory · APIs · RAG · Evaluation

Production-Ready GenAI Documentation Series

How to Use This Book

This reference document is structured so that every chapter stands alone, yet builds on preceding chapters. Each chapter contains two clearly marked sections:

Section	What You Will Find
Section 1 — Theory	Conceptual depth, mental models, diagrams, comparisons, and real-world analogies. Read this to understand why things work the way they do.
Section 2 — Practical & Coding	Annotated Python/IPython notebook code cells. Every cell is explained line-by-line so you can reproduce and adapt the examples in your own projects.

The document covers eight core topics: Tools, Structured Outputs, State Management, Short-Term Memory, External APIs & MCP, Web Search Agents, Database Interaction, Agentic RAG, Long-Term Memory, and Agent Evaluation.

Chapter 1 — Extending Agents with Tools

Language models are extraordinarily capable reasoners, but without the ability to interact with the outside world they remain confined to the information encoded in their weights at training time. Tools bridge this gap — they turn a language model from a sophisticated text predictor into an active agent that can fetch live data, execute code, write to databases, and call external services.

Section 1 — Theory

1.1 The Tool-Use Motivation

Consider the difference between a brilliant analyst who has read every book in the world but is locked in a room with no internet versus the same analyst given a computer and a phone. The raw intelligence is identical, but the scope of achievable tasks is radically different. Tools are the computer and the phone for an LLM-based agent.

Without tools, an agent attempting to answer 'What is the current EUR/USD exchange rate?' has three unappealing options:

- Refuse the question because it cannot know the live rate.
- Provide a stale rate from its training data and risk being wrong.
- Hallucinate a plausible-sounding number.

With a tool that calls a financial data API, the agent simply fetches the live rate, incorporates it into a coherent reply, and remains factually grounded. This is the fundamental value proposition of tool use.

1.2 Anatomy of a Tool

Every tool in the function-calling paradigm has four components:

Component	Description	Example
Name	A unique, descriptive identifier the model uses to reference the tool.	get_weather
Description	A natural-language explanation of what the tool does. The model reads this to decide when to call the tool.	Returns current weather for a given city.
Parameters	A JSON Schema object describing every argument — name, type, description, and which are required.	{ city: string, units: string }
Handler	The actual code (function) that executes when the model triggers the tool and returns a result.	Python def that calls OpenWeatherMap API

1.3 Common Tool Categories

- **Web Search APIs** — Query search engines for real-time information. Examples: Tavily, SerpAPI, Bing Search.
- **Math / Calculator** — Perform precise arithmetic. Eliminates hallucinated calculations for financial or scientific queries.
- **APIs & Webhooks** — Integrate with third-party services: send Slack messages, create calendar events, book flights, query CRMs.
- **Databases (SQL/NoSQL)** — Read and write structured data, enabling agents to query business records with natural language.
- **Code Execution (Python)** — Run arbitrary code for data transformation, file processing, or complex logic beyond what prompt engineering can achieve.
- **File & Document Readers** — Parse PDFs, spreadsheets, and text files, extracting structured information for downstream reasoning.

1.4 Evolution from Prompt-Hacking to Function Calling

Early tool-use implementations were brittle. Developers would instruct models to emit special strings like `[CALL: calculator(3*7)]` and then parse those strings with regex in the backend. This worked sometimes, but it was fragile: the model might format the call differently, include extra explanation, or hallucinate tool names.

Modern Function Calling is fundamentally different. The tool schema is embedded at the API level — not in the prompt — and the model is trained to produce a structured `tool_call` JSON object. The API intercepts this object before returning to the developer, making the whole interaction deterministic and type-safe.

1.5 The Function Calling Lifecycle (Step-by-Step)

Step	Actor	Action
1	Developer	Define tools as JSON schemas and include them in the API request alongside the user message.
2	Model	Read the user message, reason about which tool (if any) to call, and emit a structured <code>tool_call</code> JSON object.
3	Backend	Detect the <code>tool_call</code> , extract the function name and arguments, and execute the corresponding handler.
4	Tool	Execute (e.g., call an API, run a query) and return the result.
5	Developer	Pass the tool result back to the model as a <code>tool_result</code> message.
6	Model	Read the result and incorporate it into a final, grounded natural-language response.

Key Insight

The model never executes code. It only decides which tool to call and with what arguments. Execution always happens on the developer's backend, preserving full control and security.

1.6 Emerging Standardisation: MCP, Operator, and A2A

Just as USB-C standardised device connectivity, the industry is now standardising how agents interact with tools. Three initiatives lead this space:

- **Anthropic's Model Context Protocol (MCP)** — An open standard that defines a universal message format for tool calls, tool results, and streaming interactions. Already supported by Claude models; adoption is growing in LangChain, LangGraph, and other frameworks.
- **OpenAI's Operator** — A browser-automation agent capability that enables models to interact with web UIs as if they were human users.
- **Google's Agent-to-Agent (A2A)** — A protocol for agents to delegate sub-tasks to specialised sibling agents, enabling collaborative multi-agent pipelines.

These developments point toward a near future in which agents are modular, interoperable, and composable — each agent a specialist that plugs into a larger orchestration layer.

Section 2 — Practical & Coding

The following annotated notebook walks through building a complete tool-augmented agent from scratch. Each cell is explained in full so you understand not just what the code does but why every design decision was made.

Cell 1 — Install Dependencies

```
# Cell 1: Install required libraries
!pip install openai anthropic requests python-dotenv -q
```

Cell Explanation

openai / anthropic — SDK clients for calling LLM APIs. Either can be used; the patterns are identical.

requests — Standard HTTP library for calling external REST APIs inside our tool handlers.

python-dotenv — Loads environment variables (API keys) from a .env file, keeping secrets out of source code.

Cell 2 — Environment Setup

```
# Cell 2: Load environment variables and initialise the client
```

```

import os
from dotenv import load_dotenv
import anthropic

load_dotenv()                # Reads .env from the current directory
client = anthropic.Anthropic( # Creates a reusable API client
    api_key=os.environ['ANTHROPIC_API_KEY']
)

MODEL = 'claude-sonnet-4-20250514' # Target model; swap as needed

```

Cell Explanation

`load_dotenv()` reads key=value pairs from `.env` into `os.environ`.

The Anthropic client is instantiated once and reused across all API calls — avoiding repeated authentication overhead.

Centralising the `MODEL` constant means you change the model in one place instead of hunting through the notebook.

Cell 3 — Define the Tool Schema

```

# Cell 3: Declare the tool the model may call
tools = [
    {
        'name': 'get_weather',
        'description': ('Returns current temperature and weather conditions for a
city. '
                        'Use this when the user asks about weather in any
location.'),
        'input_schema': {
            'type': 'object',
            'properties': {
                'city': {
                    'type': 'string',
                    'description': 'The name of the city, e.g. Mumbai or São
Paulo'
                },
                'units': {
                    'type': 'string',
                    'enum': ['celsius', 'fahrenheit'],
                    'description': 'Temperature units. Default to celsius.'
                }
            }
        },
        'required': ['city']
    }
]

```

Cell Explanation

tools is a list of dictionaries. Each dictionary describes exactly one capability the model can invoke.

name — Must match the string the model outputs in its `tool_call`. Keep it snake_case and descriptive.

description — This is the most important field. The model reads it verbatim when deciding whether to call this tool vs. answering from memory. Include when to use it explicitly.

input_schema — Standard JSON Schema. 'required' lists fields the model MUST always provide. Omit a field from required to make it optional.

enum — Restricts the model to only valid values, preventing nonsense like units: 'Kelvin'.

Cell 4 — Implement the Tool Handler

```
# Cell 4: The actual function that executes when the tool is called
import requests

def get_weather(city: str, units: str = 'celsius') -> dict:
    """
    Fetches live weather data from OpenWeatherMap.
    Returns a dict the model will receive as the tool result.
    """
    api_key = os.environ['OPENWEATHERMAP_API_KEY']
    unit_code = 'metric' if units == 'celsius' else 'imperial'

    response = requests.get(
        'https://api.openweathermap.org/data/2.5/weather',
        params={'q': city, 'appid': api_key, 'units': unit_code},
        timeout=10
    )
    response.raise_for_status()  # Raises HTTPError for 4xx/5xx responses
    data = response.json()

    return {
        'city': city,
        'temperature': data['main']['temp'],
        'description': data['weather'][0]['description'],
        'humidity': data['main']['humidity'],
        'units': units,
    }
```

Cell Explanation

This function is 100% regular Python — the LLM never touches it. The agent framework calls it after inspecting the model's `tool_call`.

`timeout=10` — Always set a timeout on external HTTP calls. Without it, a slow API can freeze your agent indefinitely.

`raise_for_status()` — Converts HTTP error codes (404, 500, etc.) into Python exceptions, making error handling explicit.

The return value is a plain dict. The framework serialises it to JSON and sends it back to the model as a `tool_result` message.

The model will read every key in the returned dict, so include all context the model needs to write a complete answer.

Cell 5 — The Agent Execution Loop

```
# Cell 5: Run the agent with tool-use support
import json

def run_agent(user_message: str) -> str:
    messages = [{'role': 'user', 'content': user_message}]

    while True:
        response = client.messages.create(
            model=MODEL,
            max_tokens=1024,
            tools=tools,
            messages=messages
        )

        # Case 1: Model is done — return its final answer
        if response.stop_reason == 'end_turn':
            return response.content[0].text

        # Case 2: Model wants to call a tool
        if response.stop_reason == 'tool_use':
            tool_results = []

            for block in response.content:
                if block.type == 'tool_use':
                    tool_name = block.name
                    tool_args = block.input

                    # Dispatch to the correct handler
                    if tool_name == 'get_weather':
                        result = get_weather(**tool_args)
                    else:
                        result = {'error': f'Unknown tool: {tool_name}'}

                    tool_results.append({
                        'type': 'tool_result',
                        'tool_use_id': block.id,
                        'content': json.dumps(result),
                    })

            # Feed results back — the loop continues
            messages.append({'role': 'assistant', 'content': response.content})
            messages.append({'role': 'user', 'content': tool_results})

    # Test it
    print(run_agent('What is the weather like in Delhi right now?'))
```


Cell Explanation

`while True` loop — The agent runs until `stop_reason == 'end_turn'`. This naturally handles multi-hop scenarios where the model calls several tools before it has enough information to answer.

`stop_reason == 'tool_use'` — The model is signalling it needs external data. Never return to the user at this point.

`block.id` — Each tool call has a unique ID. You MUST echo this back in `tool_use_id` so the model can correlate the result with its request.

`messages.append(assistant content)` — The full response (including `tool_use` blocks) must be appended to the conversation before the `tool_result`. Without this, the API will reject the request.

`json.dumps(result)` — Tool results are sent as JSON strings. The model parses them and synthesises a final answer.

Chapter 2 — Structured Outputs

Conversational text is great for humans, but software systems that consume agent output need machine-readable data. Structured outputs transform agent responses from free-form prose into precise, type-safe data structures that can be directly processed by downstream systems — ticketing platforms, databases, dashboards, and orchestration pipelines.

Section 1 — Theory

2.1 Why Structure Matters

Imagine a customer service agent that reads a transcript of a support call. Two possible outputs:

Unstructured (Prose) Output	Structured (JSON) Output
"The customer seems to be experiencing a login issue. It sounds urgent and their email might be jane@mail.com."	{ "issue_type": "login_problem", "urgency": "high", "customer_email": "jane@mail.com" }
Readable by humans. Cannot be directly used by any software. Requires another model or human to extract fields.	Directly insertable into a ticketing system, CRM, or alert pipeline. Zero additional parsing required.

2.2 Why Simple Prompting Is Insufficient

A common beginner approach is to add 'Respond only in JSON with fields: issue_type, urgency, email' to the prompt. This works perhaps 70–80% of the time, but it fails in predictable ways:

- The model wraps the JSON in a markdown code block (````json ... ````), breaking `JSON.parse()`.
- The model adds explanatory prose before or after the JSON object.
- The model uses invalid values: "urgency": "very high" when the schema expects "high" | "medium" | "low".
- The model omits required fields when it cannot infer a value rather than returning null.

Each of these failures causes downstream code to crash. Production systems cannot tolerate 20–30% failure rates.

2.3 The Three Reliability Tiers

Approach	How It Works	Reliability
Prompt Engineering	Ask the model to output JSON in the system prompt.	~70–80%. Fragile in edge cases.
Output Parsers + Retry	Parse the response; on failure, re-prompt with the error message. Loop until valid.	~90–95%. Better, but adds latency.

Approach	How It Works	Reliability
Function Calling / Tool Use	The model is constrained at the API level to emit a valid tool call matching the schema.	~99%+. Near-deterministic.

2.4 Pydantic for Schema Enforcement

Pydantic is a Python library that defines data schemas as class definitions and validates incoming data against them at runtime. It integrates naturally with LLM frameworks and provides three benefits:

1. Type safety — Fields are typed (str, int, datetime, Literal). Wrong types raise a `ValidationError`.
2. Automatic coercion — Pydantic can convert "42" (string) to 42 (int) when appropriate.
3. Detailed error messages — When validation fails, Pydantic tells you exactly which field failed and why, making retry prompts precise.

A representative Pydantic model for a task-management agent:

```
from pydantic import BaseModel, EmailStr
from typing import Literal
from datetime import datetime

class SupportTicket(BaseModel):
    issue_type: Literal['login_problem', 'billing_error', 'feature_request',
                        'other']
    urgency: Literal['low', 'medium', 'high', 'critical']
    customer_email: EmailStr
    summary: str
    created_at: datetime
```

2.5 Designing for Resilience

Even with function calling, agents can occasionally produce unexpected outputs. A resilient structured-output system implements the following layers:

- Schema validation at the parser level (Pydantic or jsonschema).
- A retry loop that re-prompts with the specific validation error: 'The field urgency must be one of: low, medium, high, critical. You returned: very urgent. Please retry.'
- A fallback handler for repeated failures — log the raw response, alert a human, or return a default safe value.
- Observability logging so you can track which prompts fail most often and iteratively improve your schemas.

Section 2 — Practical & Coding

Cell 1 — Define a Pydantic Schema

```
# Cell 1: Define strict output schema with Pydantic
from pydantic import BaseModel, EmailStr, field_validator
from typing import Literal, List, Optional
from datetime import datetime

class ActionItem(BaseModel):
    title: str
    due_date: datetime
    owner: str
    priority: Literal['low', 'medium', 'high']
    status: Literal['open', 'in_progress', 'closed']

class MeetingNotes(BaseModel):
    meeting_title: str
    date: datetime
    attendees: List[str]
    summary: str
    action_items: List>ActionItem]
    follow_up_required: bool
```

Cell Explanation

BaseModel — Every Pydantic schema inherits from this. Validation runs automatically on instantiation.

Literal[...] — Restricts a field to a fixed set of valid values. The model is forced to choose one of these strings exactly.

List>ActionItem] — Nested models. Pydantic recursively validates every item in the list against the ActionItem schema.

datetime — Pydantic automatically parses ISO 8601 strings ('2025-09-15T14:00:00') into Python datetime objects.

bool — Binary field. Prevents the model from returning ambiguous strings like 'maybe' or 'depends'.

Cell 2 — Function Calling for Structured Extraction

```
# Cell 2: Use function calling to guarantee structured output
import anthropic, json
from pydantic import ValidationError

# Build the tool schema from the Pydantic model's JSON schema
extract_tool = {
    'name': 'extract_meeting_notes',
    'description': 'Extract structured meeting notes from raw transcript text.',
    'input_schema': MeetingNotes.model_json_schema() # Pydantic v2
}

def extract_notes(transcript: str, max_retries: int = 3) -> MeetingNotes:
    messages = [{'role': 'user', 'content': f'Extract meeting notes
from:\n\n{transcript}'}]
```

```

for attempt in range(max_retries):
    response = client.messages.create(
        model=MODEL,
        max_tokens=1024,
        tools=[extract_tool],
        tool_choice={'type': 'tool', 'name': 'extract_meeting_notes'},
        messages=messages
    )

    for block in response.content:
        if block.type == 'tool_use':
            try:
                return MeetingNotes(**block.input)  # Validate!
            except ValidationError as e:
                # Feed the error back for retry
                error_msg = f'Validation failed: {e}. Please fix and retry.'
                messages.append({'role': 'assistant', 'content':
                    response.content})
                messages.append({'role': 'user', 'content': [{'type':
                    'tool_result',
                        'tool_use_id': block.id, 'content': error_msg}]})
                break

    raise RuntimeError('Failed to extract valid structured output after
retries.')

```

Cell Explanation

`MeetingNotes.model_json_schema()` — Pydantic v2 generates a complete JSON Schema from the class definition. We pass this directly as the tool's `input_schema`, ensuring the model's tool call and our Pydantic class are always in sync.

`tool_choice={'type': 'tool', 'name': ...}` — Forces the model to call this specific tool. Without this, it might decide to answer in prose.

`MeetingNotes(**block.input)` — Pydantic validates and instantiates the model from the dict. If any field is wrong, a `ValidationError` is raised immediately.

Retry loop — We append the validation error as a `tool_result` message. The model reads its own mistake and corrects it on the next attempt.

`max_retries=3` — Prevents infinite loops. Three attempts are almost always sufficient for well-designed schemas.

Chapter 3 — Agent State Management

An agent that cannot remember what it just did is no better than a stateless function. For agents that execute multi-step workflows — reasoning, calling tools, receiving results, reasoning again — state is the thread that ties all of these steps together into a coherent execution.

Section 1 — Theory

3.1 Stateless vs. Stateful Systems

Stateless System	Stateful System
Every request is independent. No information flows between invocations.	Information from prior steps is preserved and available to future steps.
Example: a pure function — $f(x)$ always returns the same result.	Example: a shopping cart — it remembers what you added in earlier steps.
LLMs are inherently stateless — each API call is isolated.	Agent frameworks add state on top of stateless LLMs by injecting history into each call.

3.2 What Constitutes Agent State?

For an LLM-based agent, state is a snapshot of everything that matters at a given point in the execution. A complete state object typically contains:

- query — The original user input that initiated the task.
- instructions — The system prompt and any dynamic instructions injected by the framework.
- messages — The full conversation history: user messages, assistant responses, tool calls, and tool results.
- tool_calls — A record of which tools have been called, with what arguments, and what they returned.
- intermediate_results — Partial outputs from sub-steps that need to be aggregated.
- metadata — Timestamps, attempt counts, error logs, and any other bookkeeping fields.

Important

State is ephemeral by default — it lives in memory for the duration of a single execution. If you need state to survive restarts or be shared across workers, you must explicitly persist it to a database or message queue.

3.3 Modelling Execution as a State Machine

A state machine is a formal model in which a system exists in one of a finite set of states at any given time and transitions between states in response to inputs or conditions. Mapping an agent's execution to a state machine gives you:

4. Clarity — Every step of the agent's reasoning is named and explicit.
5. Conditional routing — Transitions only occur when the state satisfies specific conditions.
6. Debuggability — You can inspect the state at any transition point.
7. Resumability — A persisted state can be replayed from any checkpoint.

A typical agent state machine has five states:

State	What Happens	Transition Condition
PREPARE	Build the system prompt and message list from the current state.	Always → CALL_LLM
CALL_LLM	Send the messages to the model and receive a response.	Tool call detected → EXECUTE_TOOLS; else → END
EXECUTE_TOOLS	Run each requested tool and collect results.	Always → UPDATE_STATE
UPDATE_STATE	Append tool results to the message history.	Always → CALL_LLM
END	Return the final response to the caller.	Terminal — no further transitions.

Section 2 — Practical & Coding

Cell 1 — Define the State Schema with TypedDict

```
# Cell 1: Define agent state schema
from typing import TypedDict, List, Optional, Dict, Any

class AgentState(TypedDict):
    query: str # Original user question
    instructions: str # System prompt
    messages: List[Dict[str, Any]] # Full conversation history
    tool_calls: List[Dict[str, Any]] # Completed tool calls
    intermediate_data: List[Any] # Sub-step results
    final_response: Optional[str] # Populated when task is done
    error_log: List[str] # Errors encountered
    attempt_count: int # Guard against infinite loops
```

Cell Explanation

TypedDict — A lightweight way to define the shape of a dict with type annotations. Unlike Pydantic, it adds no runtime overhead — it's purely for static analysis and documentation.

List[Dict[str, Any]] — The messages field holds the entire conversation in the format expected by the Anthropic/OpenAI API.

Optional[str] — final_response is None until the agent finishes. This lets transition logic check if isinstance(state['final_response'], str) to know the task is done.

attempt_count — An essential safety valve. Increment it on every LLM call and break the loop if it exceeds a threshold (e.g., 10).

Cell 2 — State Machine Implementation

```
# Cell 2: Implement the state machine steps
import json, anthropic

MAX_ATTEMPTS = 10

def step_prepare(state: AgentState) -> AgentState:
    """Initialise messages from query and instructions if not already set."""
    if not state['messages']:
        state['messages'] = [
            {'role': 'user', 'content': state['query']}
        ]
    return state

def step_call_llm(state: AgentState) -> AgentState:
    """Call the LLM and store the raw response in the state."""
    state['attempt_count'] += 1
    response = client.messages.create(
        model=MODEL,
        max_tokens=1024,
        system=state['instructions'],
        tools=tools,
        messages=state['messages']
    )
    state['_last_response'] = response # Temporary field for routing
    return state

def step_execute_tools(state: AgentState) -> AgentState:
    """Execute all tool calls requested by the model."""
    response = state['_last_response']
    tool_results = []

    for block in response.content:
        if block.type == 'tool_use':
            result = dispatch_tool(block.name, block.input)
            state['tool_calls'].append({'tool': block.name, 'args': block.input,
            'result': result})
            tool_results.append({'type': 'tool_result', 'tool_use_id': block.id,
            'content': json.dumps(result)})

    state['messages'].append({'role': 'assistant', 'content': response.content})
    state['messages'].append({'role': 'user', 'content': tool_results})
    return state

def route(state: AgentState) -> str:
    """Decide which step to execute next."""
    if state['attempt_count'] >= MAX_ATTEMPTS:
```



```
        return 'end'
    response = state.get('_last_response')
    if response and response.stop_reason == 'tool_use':
        return 'execute_tools'
    return 'end'
```

Cell Explanation

step_prepare — Idempotent initialisation. Running it multiple times does not corrupt the state because it only sets messages when the list is empty.

_last_response — A temporary field used only by the routing function. In production systems, use a dedicated field or pass the response as a function argument.

step_execute_tools — Appends the assistant's response (including `tool_use` blocks) before appending tool results. This ordering is required by the Anthropic API.

route — The conditional dispatch function. It reads the state and returns a string that maps to the next step. This makes the flow explicit and testable.

MAX_ATTEMPTS guard — Without this, a misconfigured tool or a looping model could run indefinitely, burning tokens and money.

Chapter 4 — Short-Term Agent Memory

LLMs are amnesiac by design — each API call begins with a blank slate. Short-term memory is the engineering solution that gives agents the appearance of continuous awareness within a session, enabling them to build on earlier exchanges rather than starting fresh with every turn.

Section 1 — Theory

4.1 The Three Memory Scopes

Type	Scope	Keyed By	Lifetime
Execution State	One task run	run_id	Duration of the run
Short-Term Memory	One user session	session_id	Until the browser closes / session expires
Long-Term Memory	Across sessions	user_id	Indefinite (with TTL policies)

4.2 Memory Simulation Strategies

Because LLMs have no internal memory, all three types are simulated by injecting prior context into the prompt. Short-term memory specifically uses one of three strategies:

Strategy	How It Works	Trade-offs
Full History	Send every message from the session in every API call.	Perfect recall. Token count grows unboundedly — expensive and eventually hits the context window limit.
Sliding Window	Keep only the N most recent messages (e.g., last 20).	Bounded token cost. Early context is lost — can confuse the agent if the user refers back to it.
Summarisation	Periodically summarise older messages into a compact paragraph; keep the summary + recent messages.	Efficient. Summary quality depends on the summarisation model. Some nuance is inevitably lost.

4.3 State vs. Session Memory — The Distinction

These two concepts are frequently conflated but are conceptually distinct:

- Execution State** — What the agent is doing right now during a single task invocation. It includes tool calls in flight, intermediate results, and routing metadata. It disappears when the task ends.

- **Session Memory** — What was discussed across multiple task invocations within the same user session. It is the accumulated history of turns, not a single run's working memory.

A well-designed agent maintains both: state keeps the current run coherent, and session memory keeps the multi-turn dialogue coherent.

Section 2 — Practical & Coding

Cell 1 — Full History Manager

```
# Cell 1: Simple full-history session manager
from dataclasses import dataclass, field
from typing import List, Dict, Any

class FullHistorySession:
    def __init__(self, system_prompt: str = ''):
        self.system_prompt = system_prompt
        self.messages: List[Dict[str, Any]] = []

    def add_user(self, content: str):
        self.messages.append({'role': 'user', 'content': content})

    def add_assistant(self, content):
        self.messages.append({'role': 'assistant', 'content': content})

    def chat(self, user_input: str) -> str:
        self.add_user(user_input)
        response = client.messages.create(
            model=MODEL,
            max_tokens=512,
            system=self.system_prompt,
            messages=self.messages
        )
        reply = response.content[0].text
        self.add_assistant(reply)
        return reply

# Usage
session = FullHistorySession('You are a helpful data science tutor.')
print(session.chat('What is gradient descent?'))
print(session.chat('Can you show me a Python example?')) # Remembers Q1
```

Cell Explanation

`FullHistorySession` encapsulates the message list so callers never need to manage it manually. `add_user` / `add_assistant` — Separate helpers ensure messages are appended in the correct role order.

The second `chat()` call includes the full prior conversation in messages. The model 'remembers' Q1 because it reads it — not because it has internal memory.

Limitation: After ~100 turns, the message list will exceed the context window. Switch to sliding window or summarisation at that point.

Cell 2 — Sliding Window Session

```
# Cell 2: Sliding window memory — bounded token cost
class SlidingWindowSession:
    def __init__(self, system_prompt: str = '', window_size: int = 10):
        self.system_prompt = system_prompt
        self.window_size = window_size      # Max messages to retain
        self._all_messages: List[Dict] = [] # Full history (for audit)

    def _windowed(self) -> List[Dict]:
        """Return only the last window_size messages."""
        return self._all_messages[-self.window_size:]

    def chat(self, user_input: str) -> str:
        self._all_messages.append({'role': 'user', 'content': user_input})
        response = client.messages.create(
            model=MODEL,
            max_tokens=512,
            system=self.system_prompt,
            messages=self._windowed()      # Send only the window!
        )
        reply = response.content[0].text
        self._all_messages.append({'role': 'assistant', 'content': reply})
        return reply
```

Cell Explanation

`_all_messages` stores the complete history for logging and auditing purposes — you never lose it.

`_windowed()` slices the last `N` messages. `window_size=10` means the model sees at most 5 user turns and 5 assistant turns.

The window is computed fresh on every call, so you can dynamically adjust `window_size` at runtime without affecting `_all_messages`.

Best practice: Pair this with a summarisation step that periodically condenses the oldest messages outside the window into a 'Prior context: ...' prefix in the system prompt.

Chapter 5 — External Tools, APIs & Model Context Protocol

An agent that can only reason is an advisor. An agent that can call external APIs is an actor. This chapter covers the mechanics of integrating real-world services into agents and introduces the Model Context Protocol (MCP) — the emerging standard that makes these integrations portable, auditable, and interoperable.

Section 1 — Theory

5.1 The REST API Mental Model

Most external services expose a REST API: a collection of HTTP endpoints that accept structured requests and return structured responses, typically in JSON. Understanding REST is prerequisite knowledge for any agent engineer.

HTTP Method	Operation	When to Use
GET	Read	Fetch data without changing anything (weather, stock prices, user records).
POST	Create	Submit new data (book a flight, create a ticket, send a message).
PUT/PATCH	Update	Modify existing data (update ticket status, change a booking).
DELETE	Remove	Delete a record. Use with extreme caution in agents — irreversible.

5.2 Authentication Patterns

Almost every production API requires authentication. Agent engineers must handle two common patterns:

- **API Keys** — A static secret string passed in the request header (X-API-Key) or as a query parameter. Simple but offers no per-user granularity. Store in environment variables, never in code.
- **OAuth 2.0** — A delegation protocol for user-specific access. The user authorises the agent, receives a short-lived access token, and the agent uses that token. Required for Google Calendar, Gmail, GitHub, and most user-data services.

Security Rule

Never hardcode API keys in source code or Jupyter notebooks.
Always load them from environment variables (.env files or secrets managers like AWS Secrets Manager).
Rotate keys periodically and revoke them immediately if a repository is accidentally made public.

5.3 Fault Tolerance for API Calls

External APIs fail. Networks are unreliable. Rate limits exist. An agent that crashes on the first API failure is not production-ready. The following strategies build resilience:

8. **Timeouts** — Set a maximum wait time for every request (e.g., 10 seconds). Never call an API without one.
9. **Retry with Exponential Backoff** — On transient failures (503, 429), wait 1s, then 2s, then 4s before retrying. Avoids hammering an already-stressed service.
10. **Circuit Breaker** — After N consecutive failures, stop calling the service and return a graceful error. Prevents cascading failures in multi-agent systems.
11. **Fallbacks** — If the primary API is unavailable, fall back to a cached value, a secondary provider, or a polite 'I cannot retrieve that right now' response.

5.4 Model Context Protocol (MCP) — Deep Dive

MCP is an open standard, proposed and maintained by Anthropic, that defines how LLMs interact with tools in a model-agnostic, vendor-neutral way.

The core MCP architecture separates concerns into three layers:

- **Intent Layer** — The model declares what it wants: 'I want to call tool X with arguments Y'. This is expressed in a standard message format, regardless of which model or vendor is involved.
- **Execution Layer** — The MCP server handles the actual API call: authentication, retries, logging, rate limiting. The model never touches credentials.
- **Streaming Layer** — For long-running tools, MCP supports Streamable HTTP (replacing the deprecated SSE transport as of March 2025), enabling the server to stream partial results back to the model.

MCP vs. Direct Function Calling

Direct Function Calling: You write a handler for every tool in your application code. Works well for single-model, single-vendor deployments.

MCP: Tools are exposed by independent MCP servers. Any MCP-compatible model can use any MCP-compatible tool without changes to the agent code.

Think of MCP as the USB-C of AI tool integration — one standard connector for every device.

Section 2 — Practical & Coding

Cell 1 — Robust API Caller with Retry

```
# Cell 1: Production-grade API caller with exponential backoff
import requests, time, logging
from typing import Optional, Dict, Any
```

```

logger = logging.getLogger(__name__)

def api_call_with_retry(
    url: str,
    method: str = 'GET',
    headers: Optional[Dict] = None,
    params: Optional[Dict] = None,
    json: Optional[Dict] = None,
    max_retries: int = 3,
    backoff_base: float = 1.0,
    timeout: int = 10
) -> Dict[str, Any]:
    for attempt in range(max_retries):
        try:
            response = requests.request(
                method=method, url=url, headers=headers,
                params=params, json=json, timeout=timeout
            )
            if response.status_code == 429:          # Rate limited
                wait = backoff_base * (2 ** attempt)
                logger.warning(f'Rate limited. Waiting {wait}s before retry
{attempt+1}.')
                time.sleep(wait)
                continue
            response.raise_for_status()
            return response.json()
        except requests.Timeout:
            logger.error(f'Timeout on attempt {attempt+1} for {url}')
            if attempt == max_retries - 1:
                raise
            time.sleep(backoff_base * (2 ** attempt))
        except requests.HTTPError as e:
            if response.status_code < 500:
                raise # Client errors (4xx) are not retryable
            time.sleep(backoff_base * (2 ** attempt))
    raise RuntimeError(f'All {max_retries} attempts failed for {url}')

```

Cell Explanation

`status_code == 429` — Rate limit response. We wait with exponential backoff rather than retrying immediately.

`backoff_base * (2 ** attempt)` — Exponential backoff: waits 1s, 2s, 4s on successive attempts. Adds no extra load to an already-stressed service.

`if response.status_code < 500: raise` — 4xx errors (bad request, not found, unauthorised) are client errors. Retrying them will never succeed; raise immediately.

`requests.Timeout` — Separates timeout handling from HTTP error handling. A timeout might be transient (temporary network issue), so we retry it.

This function is a reusable building block. Every tool handler in your agent should call it instead of raw `requests.get()`.

Chapter 6 — Web Search Agents

The web is the largest, most up-to-date knowledge base in existence — but LLMs cannot access it after their training cutoff. Web search agents close this gap, enabling agents to retrieve current information, ground their responses in real evidence, and cite sources rather than hallucinating.

Section 1 — Theory

6.1 Why Web Search Matters

Without Web Search	With Web Search
Agent's knowledge is frozen at training cutoff.	Agent retrieves live information at query time.
Risk of confidently stating stale facts.	Can cite sources, reducing hallucination risk.
Useless for queries about recent events.	Handles news, product launches, live scores, etc.

6.2 Common Web Search APIs

- **Tavily** — Designed specifically for LLM agents. Returns clean, parsed text content rather than raw HTML. Optimal for information retrieval tasks.
- **SerpAPI** — Emulates real Google search results. Supports structured result types: organic, knowledge panel, maps, images.
- **Bing Search API** — Microsoft's offering. Robust, scalable, and well-documented. Available through Azure Cognitive Services.
- **DuckDuckGo (unofficial)** — No API key required. Good for prototyping. Rate-limited and not suitable for production at scale.

6.3 Grounding and Citation

Grounding means anchoring the agent's response to specific, retrievable sources. A grounded response looks like: 'According to a Reuters article published on 14 May 2025, Nvidia announced...' rather than 'Nvidia announced...' with no attribution.

Grounding serves three purposes:

12. Accuracy — Forces the agent to base claims on real retrieved content rather than parametric memory.
13. Trust — Users can verify claims by following cited links.
14. Auditability — In regulated industries, grounded outputs are required for compliance.

6.4 Dealing with Noise

The web contains misinformation, outdated content, SEO spam, and advertising. Search-enabled agents must be designed to filter this noise:

- Prefer established, authoritative sources (Reuters, government sites, official company blogs).

- Check publication dates — a 2019 article about 'the latest iPhone' is useless for a 2025 query.
- Cross-validate claims across multiple results before stating them as fact.
- Instruct the model in the system prompt to respond with 'I could not find reliable information' rather than hallucinating when results are poor.

Section 2 — Practical & Coding

Cell 1 — Web Search Tool with Tavily

```
# Cell 1: Web search tool using Tavily API
import os, requests

TAVILY_API_KEY = os.environ['TAVILY_API_KEY']

def web_search(query: str, max_results: int = 5) -> list[dict]:
    """
    Search the web via Tavily and return structured results.
    Each result has: title, url, content (snippet), score.
    """
    response = requests.post(
        'https://api.tavily.com/search',
        json={
            'api_key': TAVILY_API_KEY,
            'query': query,
            'max_results': max_results,
            'search_depth': 'basic',      # 'advanced' for deeper scraping
            'include_answer': False,      # We want raw results, not Tavily's
summary
        },
        timeout=15
    )
    response.raise_for_status()
    return response.json().get('results', [])
```

Cell Explanation

`search_depth='basic'` — Faster and cheaper. Use 'advanced' when you need Tavily to fully parse and extract content from linked pages.

`include_answer=False` — We want the raw search results so our agent can reason over them and form its own grounded answer with citations.

`max_results=5` — 5 results is a good balance between context richness and token budget. More than 10 usually adds noise rather than value.

The function returns a list of dicts. Each dict contains title, url, content (a clean text snippet), and a relevance score.

Cell 2 — Grounded Search Agent

```

# Cell 2: Agent that searches and cites sources
import json

search_tool_schema = {
    'name': 'web_search',
    'description': ('Search the web for current information. '
                   'Use for recent events, news, prices, or any time-sensitive
data.'),
    'input_schema': {
        'type': 'object',
        'properties': {
            'query': {'type': 'string', 'description': 'The search query.'}
        },
        'required': ['query']
    }
}

SYSTEM = """You are a research assistant. When you use web_search, always:
1. Cite the source URL for every factual claim.
2. Mention the publication date if visible in the snippet.
3. If results are ambiguous, say so rather than speculating."""

def grounded_search_agent(question: str) -> str:
    messages = [{'role': 'user', 'content': question}]
    while True:
        response = client.messages.create(
            model=MODEL, max_tokens=1024,
            system=SYSTEM, tools=[search_tool_schema], messages=messages
        )
        if response.stop_reason == 'end_turn':
            return response.content[0].text
        if response.stop_reason == 'tool_use':
            tool_results = []
            for block in response.content:
                if block.type == 'tool_use':
                    results = web_search(block.input['query'])
                    tool_results.append({'type': 'tool_result',
                                        'tool_use_id': block.id,
                                        'content': json.dumps(results)})
            messages.append({'role': 'assistant', 'content': response.content})
            messages.append({'role': 'user', 'content': tool_results})

```

Chapter 7 — Agentic Retrieval-Augmented Generation

Basic RAG is a passive lookup: accept a query, retrieve documents, generate an answer. Agentic RAG transforms this into an active reasoning cycle: the agent evaluates whether what it retrieved is sufficient, identifies gaps, reformulates queries, and retries — all before composing a final answer. This chapter explains why this matters and how to build it.

Section 1 — Theory

7.1 The Limitations of Naive RAG

Naive RAG Behaviour	Agentic RAG Behaviour
Retrieves once with the original query.	May retrieve multiple times with progressively refined queries.
Uses retrieved documents regardless of relevance.	Evaluates relevance before using retrieved content.
Generates an answer even when retrieval fails.	Identifies gaps and re-queries rather than hallucinating.
One-shot: no feedback loop.	Iterative: reflection drives the next retrieval action.

7.2 The Agentic RAG Reasoning Loop

The key insight is that retrieval should be a tool the agent calls when it decides it needs more information — not an automatic pre-processing step. The loop looks like this:

- 15. Receive the user query.
- 16. Reason: Do I already know enough to answer accurately? If yes, answer. If no, proceed.
- 17. Formulate a retrieval query (which may differ from the original user question).
- 18. Retrieve documents and evaluate: Are they relevant? Do they contain the needed information?
- 19. If yes: synthesise an answer with citations. If no: reformulate and return to step 3.
- 20. Repeat until sufficient evidence is gathered or a maximum iteration limit is reached.

7.3 Query Reformulation Strategies

Effective query reformulation is the core skill that makes agentic RAG superior. Common techniques include:

- **Decomposition** — Break a complex question into sub-questions. 'Why did Zillow's iBuying program fail?' becomes 'What algorithmic assumptions did Zillow make?' + 'How did market conditions in 2021 differ from the model's training data?'
- **Expansion** — Add synonyms or related terms. 'ML model training' → 'neural network training', 'deep learning optimisation'.

- **Abstraction** — Move from specific to general when specific queries return nothing. 'Zillow iBuying losses Q3 2021' → 'iBuying business model risks'.
- **Perspective shift** — Change the angle. Instead of searching for the company's explanation, search for analyst critiques or academic papers.

Section 2 — Practical & Coding

Cell 1 — Document Retriever Tool

```
# Cell 1: Set up a simple in-memory vector store for RAG
!pip install chromadb sentence-transformers -q

import chromadb
from chromadb.utils.embedding_functions import
SentenceTransformerEmbeddingFunction

# Initialise Chroma with a local embedding model
embed_fn = SentenceTransformerEmbeddingFunction(model_name='all-MiniLM-L6-v2')
chroma = chromadb.Client()
collection = chroma.create_collection('knowledge_base',
embedding_function=embed_fn)

# Index some documents
documents = [
    'Zillow launched its iBuying program in 2018, using an algorithm to price homes.',
    'The algorithm relied on historical price trends and failed to account for rapid 2021 appreciation.',
    'Zillow wrote down $304 million in inventory losses and exited iBuying in November 2021.',
    'Analysts cited feedback loops in the algorithm as a key technical failure.',
]
collection.add(
    documents=documents,
    ids=[f'doc_{i}' for i in range(len(documents))]
)

def retrieve(query: str, n_results: int = 3) -> list[str]:
    results = collection.query(query_texts=[query], n_results=n_results)
    return results['documents'][0] # List of matched document texts
```

Cell Explanation

SentenceTransformerEmbeddingFunction — Embeds text using a lightweight local model (all-MiniLM-L6-v2). No API key needed; runs entirely on CPU.

collection.add() — Indexes each document by computing its embedding and storing it. Chroma handles the vector index automatically.

`collection.query()` — Performs a semantic similarity search. Returns the N documents whose embeddings are closest to the query embedding in vector space.

`n_results=3` — Retrieve 3 documents. More documents increase recall but also increase the prompt length and cost.

In production, replace the in-memory Chroma client with a persistent one (`chromadb.PersistentClient`) or a managed service like Pinecone or Weaviate.

Cell 2 — Agentic RAG Agent

```
# Cell 2: Agentic RAG with reflection and retry
retrieve_tool = {
    'name': 'retrieve_documents',
    'description': ('Search the knowledge base for relevant information. '
                   'Call this when you need factual grounding. '
                   'You may call it multiple times with different queries if
initial results are insufficient.'),
    'input_schema': {
        'type': 'object',
        'properties': {
            'query': {'type': 'string', 'description': 'The retrieval query.'},
            'reason': {'type': 'string', 'description': 'Why you need this
information.'}
        },
        'required': ['query', 'reason']
    }
}

RAG_SYSTEM = """You are a research analyst with access to a knowledge base.
Strategy:
- Before answering, always retrieve relevant documents.
- After retrieval, evaluate: do the documents fully answer the question?
- If not, retrieve again with a more specific or different query.
- Ground every claim in retrieved evidence. Cite the source text.
- Maximum 4 retrieval calls per question."""

def agentic_rag(question: str) -> str:
    messages = [{'role': 'user', 'content': question}]
    retrieval_count = 0
    while True:
        response = client.messages.create(
            model=MODEL, max_tokens=1024,
            system=RAG_SYSTEM, tools=[retrieve_tool], messages=messages
        )
        if response.stop_reason == 'end_turn':
            return response.content[0].text
        for block in response.content:
            if block.type == 'tool_use' and retrieval_count < 4:
                docs = retrieve(block.input['query'])
                retrieval_count += 1
                messages.append({'role': 'assistant', 'content':
response.content})
                messages.append({'role': 'user', 'content': [{'type':
'tool_result',
                        'tool_use_id': block.id, 'content': '\n\n'.join(docs)}]})
```

```
        break
    elif retrieval_count >= 4:
        return 'Maximum retrieval attempts reached.'
```

Cell Explanation

reason field in tool schema — Forces the model to articulate why it is retrieving, which improves the quality of its query formulation and creates a natural audit trail.

RAG_SYSTEM prompt — The four-step strategy is explicit. Without this guidance, the model may retrieve once and answer regardless of quality.

retrieval_count < 4 guard — Prevents the model from looping indefinitely. Four retrievals are almost always sufficient.

The retrieved documents are joined with double newlines and returned as a tool_result. The model reads all three documents and synthesises them.

This pattern handles the Zillow case study from the notes: the agent first retrieves general facts, then identifies the gap (technical root causes), then reformulates to retrieve algorithm-specific information.

Chapter 8 — Agent Evaluation

Building a capable agent is only half the problem. Knowing whether it is performing well, safely, and efficiently requires rigorous evaluation. This chapter establishes a systematic framework for measuring what matters across the full dimensions of agent behaviour.

Section 1 — Theory

8.1 Why Evaluation Cannot Be an Afterthought

Without evaluation, agent failures are invisible until they become expensive. An agent might:

- Consistently pick the wrong tool, wasting tokens on unnecessary API calls.
- Return correct answers via an incorrect reasoning path, making it fragile to paraphrasing.
- Gradually degrade as the knowledge base is updated and retrieval quality drops.
- Take harmful actions — deleting records instead of querying them — with no detection mechanism.

Evaluation provides the feedback loop that makes iterative improvement possible. Without it, you are flying blind.

8.2 Four Dimensions of Evaluation

Dimension	Key Questions	Metrics
Task Completion	Did the agent achieve the goal? How many steps?	Success rate, step count, human intervention rate
Quality Control	Was the output correct format? Followed instructions?	Schema validation rate, instruction adherence
Tool Interaction	Right tool? Valid arguments? Correct result used?	Tool selection accuracy, argument validity rate
System Efficiency	Token usage, latency, failure rates	Avg tokens/task, p95 latency, error rate

8.3 Three Evaluation Strategies

The right evaluation strategy depends on what you need to know and the complexity of your agent:

- **1. Final Response Evaluation (Black Box)** — Evaluate only the end output against a reference answer. Fast and simple. Best for: regression testing, product demos, high-level quality checks. Limitation: tells you nothing about how the agent reached its answer.

- **2. Single-Step Evaluation** — Evaluate individual decisions in isolation: 'Given this state, did the agent pick the right tool?' Best for: debugging specific failure modes, unit testing agent logic. Limitation: does not capture interaction effects across steps.
- **3. Trajectory Evaluation** — Evaluate the full execution path: every tool call, in order, with its arguments and results. Most informative. Best for: complex multi-step workflows, safety audits. Limitation: requires storing and processing full execution traces.

8.4 LLM-as-Judge

For tasks where exact match against a reference answer is impossible (open-ended questions, creative tasks, nuanced reasoning), an LLM can serve as the evaluator. The judge model reads the agent's response and rates it on dimensions like accuracy, completeness, and relevance.

Caution

LLM-as-judge has known biases: it tends to prefer longer, more confident-sounding answers even when shorter ones are more accurate.

Use structured rubrics with explicit scoring criteria rather than asking for a general 'is this good?' judgement.

Validate your LLM judge by comparing its ratings to human annotations on a sample of cases.

Section 2 — Practical & Coding

Cell 1 — Evaluation Dataset Structure

```
# Cell 1: Define an evaluation dataset
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class EvalCase:
    case_id: str
    user_input: str
    expected_tools: List[str]
    reference_answer: Optional[str]
    metadata: dict

    # The query sent to the agent
    # Tools that SHOULD be called
    # Ground truth answer (if available)
    # Tags: difficulty, category, etc.

eval_dataset = [
    EvalCase(
        case_id='case_001',
        user_input='What is the current weather in Mumbai?',
        expected_tools=['get_weather'],
        reference_answer=None,
        metadata={'category': 'weather', 'difficulty': 'easy'}
    ),
    EvalCase(
        case_id='case_002',
        user_input='What is 2+2?',
```



```

        expected_tools=[],                                # Should answer without any tool
        reference_answer='4',
        metadata={'category': 'math', 'difficulty': 'trivial'}
    ),
]

```

Cell Explanation

EvalCase dataclass — A typed, self-documenting container for evaluation examples. Using a dataclass instead of a plain dict prevents typos in field names.

expected_tools — An empty list signals that the agent should answer without calling any tool. Calling a tool unnecessarily is itself a failure mode.

reference_answer=None — Some queries have dynamic answers (live weather). For these, we evaluate tool selection and response quality, not exact text match.

metadata — Tags allow you to slice evaluation results: 'Are failures concentrated in the high-difficulty category?' 'Is the agent weak on weather queries specifically?'

Cell 2 — Automated Evaluation Runner

```

# Cell 2: Run evaluation and compute metrics
import json
from dataclasses import dataclass

@dataclass
class EvalResult:
    case_id: str
    tools_called: List[str]
    tool_match: bool # Did it call the right tools?
    final_response: str
    exact_match: Optional[bool]
    token_usage: int

def evaluate_agent(agent_fn, dataset: List[EvalCase]) -> List[EvalResult]:
    results = []
    for case in dataset:
        tools_called = []

        # Monkey-patch to intercept tool calls
        original_dispatch = dispatch_tool
        def tracking_dispatch(name, args):
            tools_called.append(name)
            return original_dispatch(name, args)

        response = agent_fn(case.user_input)
        tool_match = sorted(tools_called) == sorted(case.expected_tools)
        exact = (response.strip().lower() ==
        case.reference_answer.strip().lower()
            if case.reference_answer else None)

        results.append(EvalResult(
            case_id=case.case_id,

```

```

        tools_called=tools_called,
        tool_match=tool_match,
        final_response=response,
        exact_match=exact,
        token_usage=0 # Placeholder; extract from response object in
practice
    ))
    return results

def print_eval_summary(results: List[EvalResult]):
    total = len(results)
    tool_acc = sum(r.tool_match for r in results) / total
    exact_cases = [r for r in results if r.exact_match is not None]
    exact_acc = sum(r.exact_match for r in exact_cases) / len(exact_cases) if
exact_cases else None
    print(f'Total cases:          {total}')
    print(f'Tool accuracy:        {tool_acc:.1%}')
    if exact_acc is not None:
        print(f'Exact match acc:    {exact_acc:.1%}')

```

Cell Explanation

tools_called list — Captures every tool the agent invokes during a run. Comparing this to **expected_tools** gives tool selection accuracy.

sorted(tools_called) == sorted(case.expected_tools) — Sorting removes ordering sensitivity. We care that the right tools were called, not that they were called in a specific order.

exact_match — Only evaluated when a **reference_answer** exists. Skipped for dynamic-answer queries.

print_eval_summary — Aggregates results into the two most important headline metrics: tool accuracy and exact match. Add latency and token cost metrics in production.

Run this evaluation suite after every prompt change, model upgrade, or tool addition to catch regressions early.

Quick Reference — Patterns & Checklists

Agent Engineering Checklist

Use this checklist before shipping any agent to production:

- Tools: Every tool has a name, clear description, and JSON schema with required fields marked.
- Tools: Tool handlers have timeouts, retry logic, and graceful error handling.
- Structured Outputs: Pydantic schemas are defined for all machine-consumed outputs.
- Structured Outputs: A retry loop handles `ValidationError` with the error message fed back to the model.
- State: The state schema is documented with `TypedDict` or a dataclass.
- State: `MAX_ATTEMPTS` guard prevents infinite execution loops.
- Memory: A session manager (full history or sliding window) is in place for multi-turn agents.
- Memory: Token budget is monitored; summarisation is available for long sessions.
- APIs: API keys are loaded from environment variables, never hardcoded.
- APIs: All external calls use exponential backoff and circuit breakers.
- RAG: Retrieval is a tool, not a pre-processing step. The agent decides when to retrieve.
- RAG: Retrieval loops have a maximum iteration cap (e.g., 4).
- Evaluation: An eval dataset exists with `expected_tools` and reference answers.
- Evaluation: Evaluation runs automatically on every code change (CI/CD).
- Observability: Every tool call is logged with arguments, results, and latency.

Memory Strategy Selection Guide

If your agent...	Use...	Because...
Has sessions < 20 turns	Full History	Simple, zero complexity, perfect recall.
Has sessions > 20 turns	Sliding Window	Bounded token cost with recent-context preservation.
Needs to recall early turns	Summarisation	Compresses old context without losing key facts.
Serves returning users	Long-Term Memory (DB)	Facts persist across sessions, enabling personalisation.

Evaluation Strategy Selection Guide

Goal	Strategy	When to Use
Catch end-to-end regressions	Final Response Evaluation	After every deployment; quick smoke test.
Debug a specific failure	Single-Step Evaluation	When you know which step is broken.
Audit complex workflows	Trajectory Evaluation	For multi-step agents in regulated contexts.
Open-ended quality assessment	LLM-as-Judge	When exact reference answers don't exist.